



Politecnico
di Torino

Generative Artificial Intelligence for graphics and multimedia

ML Agents
Prof. Lia Morra



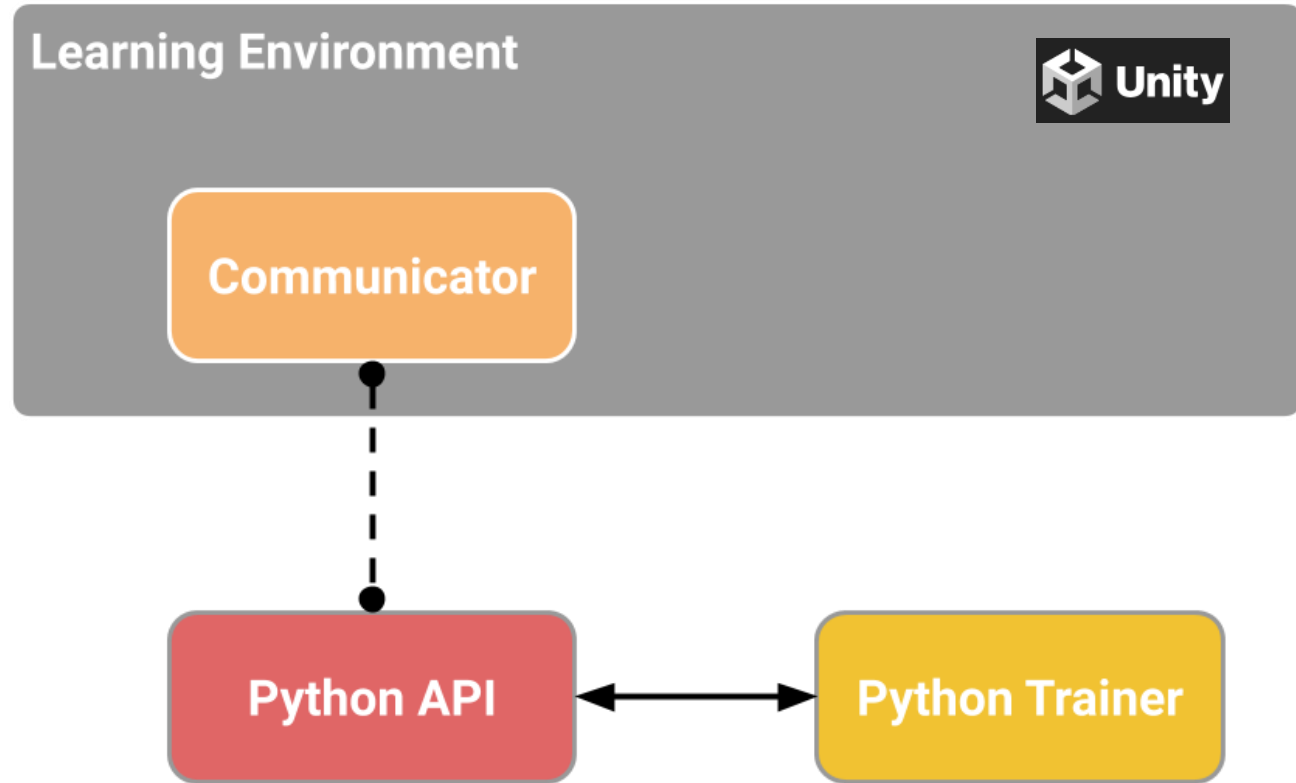
History

- **Open-source** toolkit first released by Unity Technologies in 2017
- One of the first serious attempts to make a production-grade game engine into a first-class environment for training deep reinforcement learning agents.
- Several **major versions** — early versions used a very different communication model — and as of the current ML-Agents release the architecture is mature, well-documented, and widely adopted both in academic research and in game studios.
- The community has contributed a **large library** of example environments, from simple ball-rolling tasks to multi-agent soccer

ML-Agents high-level architecture

ML-Agents splits responsibility across two languages and two processes:

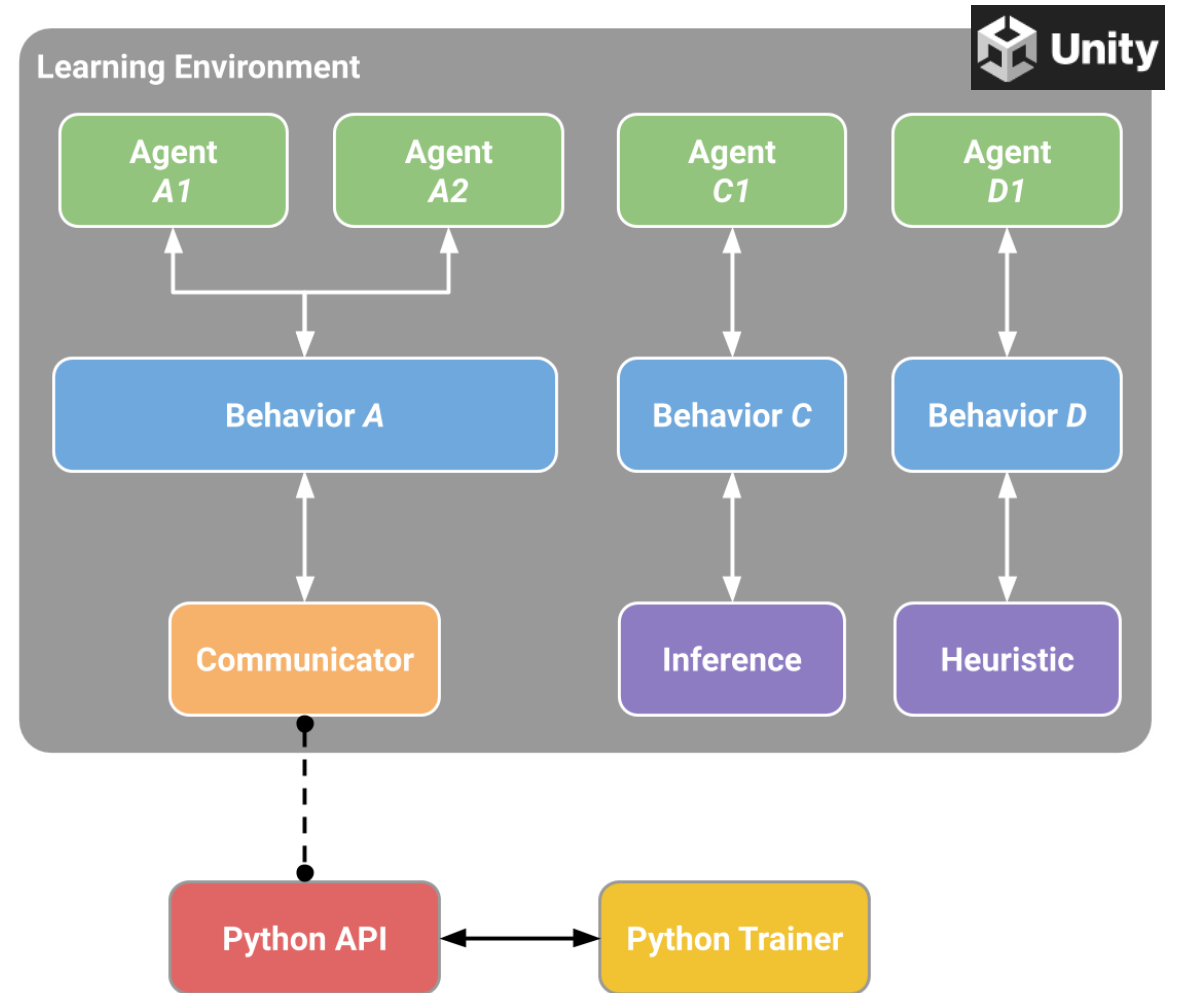
- On the **Unity** side, written in C#, the learning environment.
- On the **Python** side, the Trainer — the neural network and the training algorithm.
- These two sides communicate with each other over *gRPC*.



ML-Agents high-level architecture

Three key components:

- **Agent**
- **Behavior**
- **Environment**



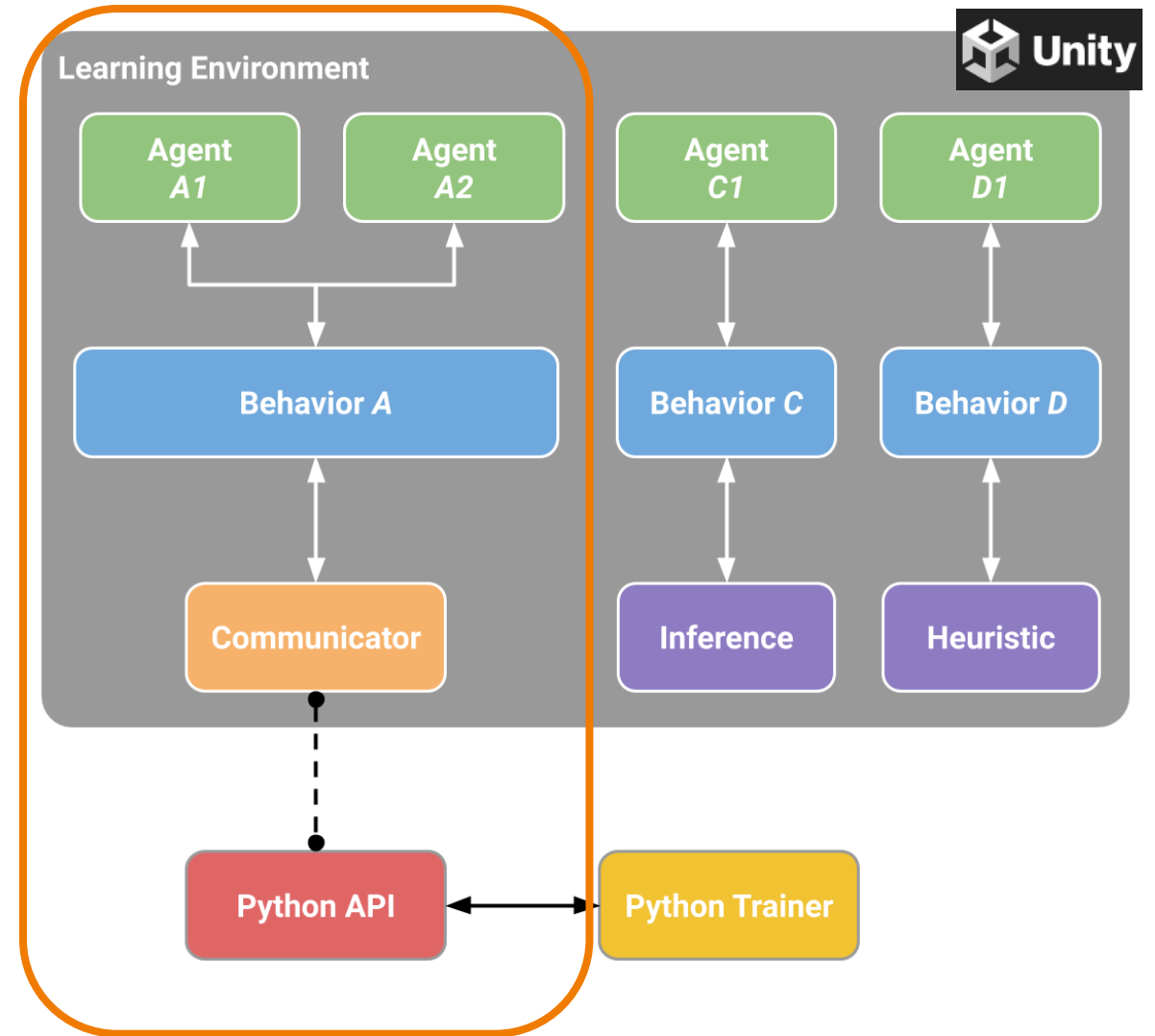
ML-Agents high-level architecture

The **Agent** is a MonoBehaviour script that you attach to any GameObject.

It is responsible for:

- collecting observations,
- receiving actions from the policy,
- computing rewards.

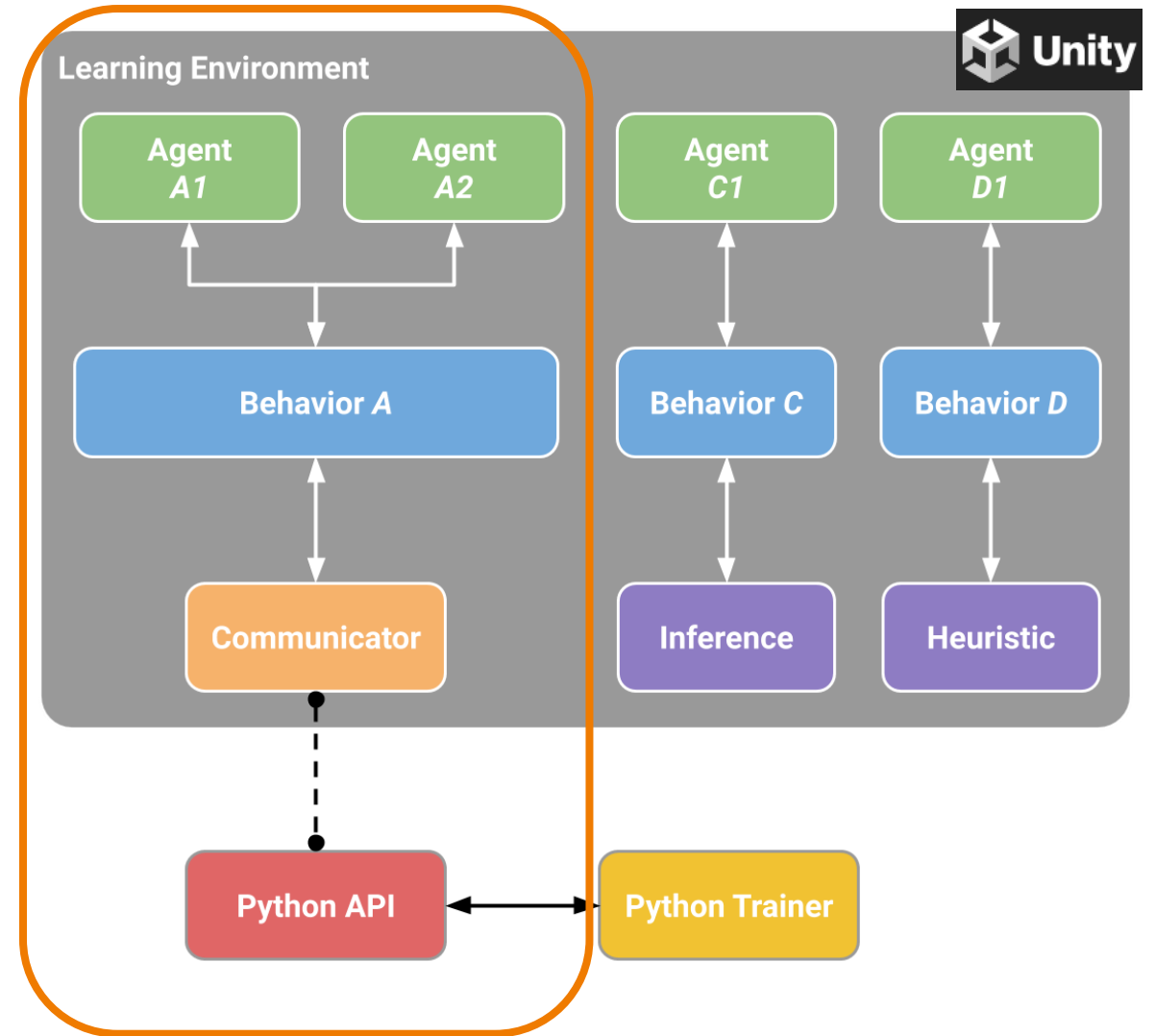
Every frame of the simulation, the Agent is either collecting data to send to Python, or receiving an action from Python and executing it



ML-Agents high-level architecture

The **Policy** class abstracts out the decision-making logic from the Agent itself so that you can use the same Policy in multiple Agents.

- How a Policy makes its decisions depends on the Behavior Parameters associated with the agent.
- If the Agent has a `Model` file, its Policy will use the neural network `Model` to take decisions.

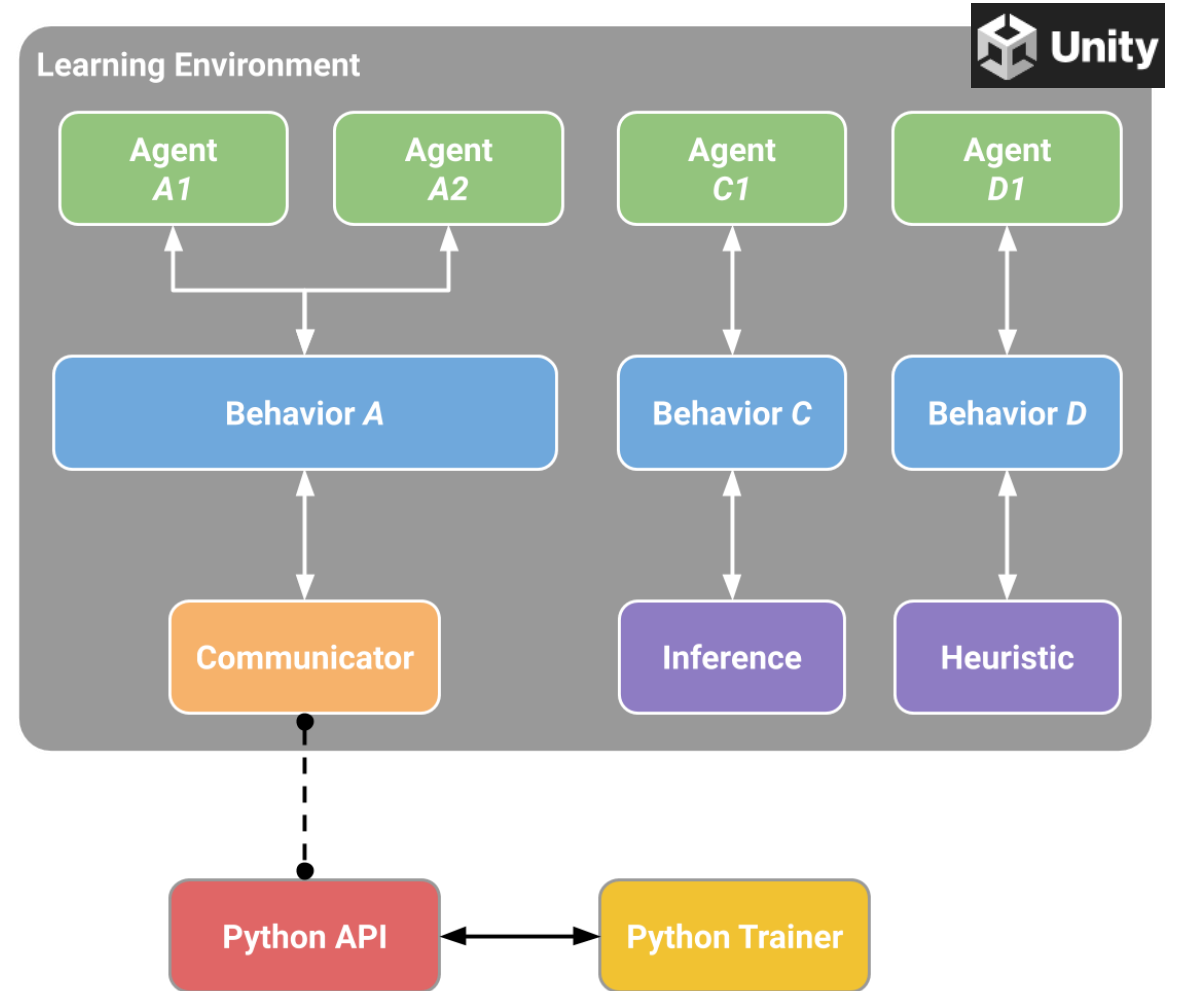


ML-Agents high-level architecture

The **Behavior** is a configuration object that links the Agent to a particular policy.

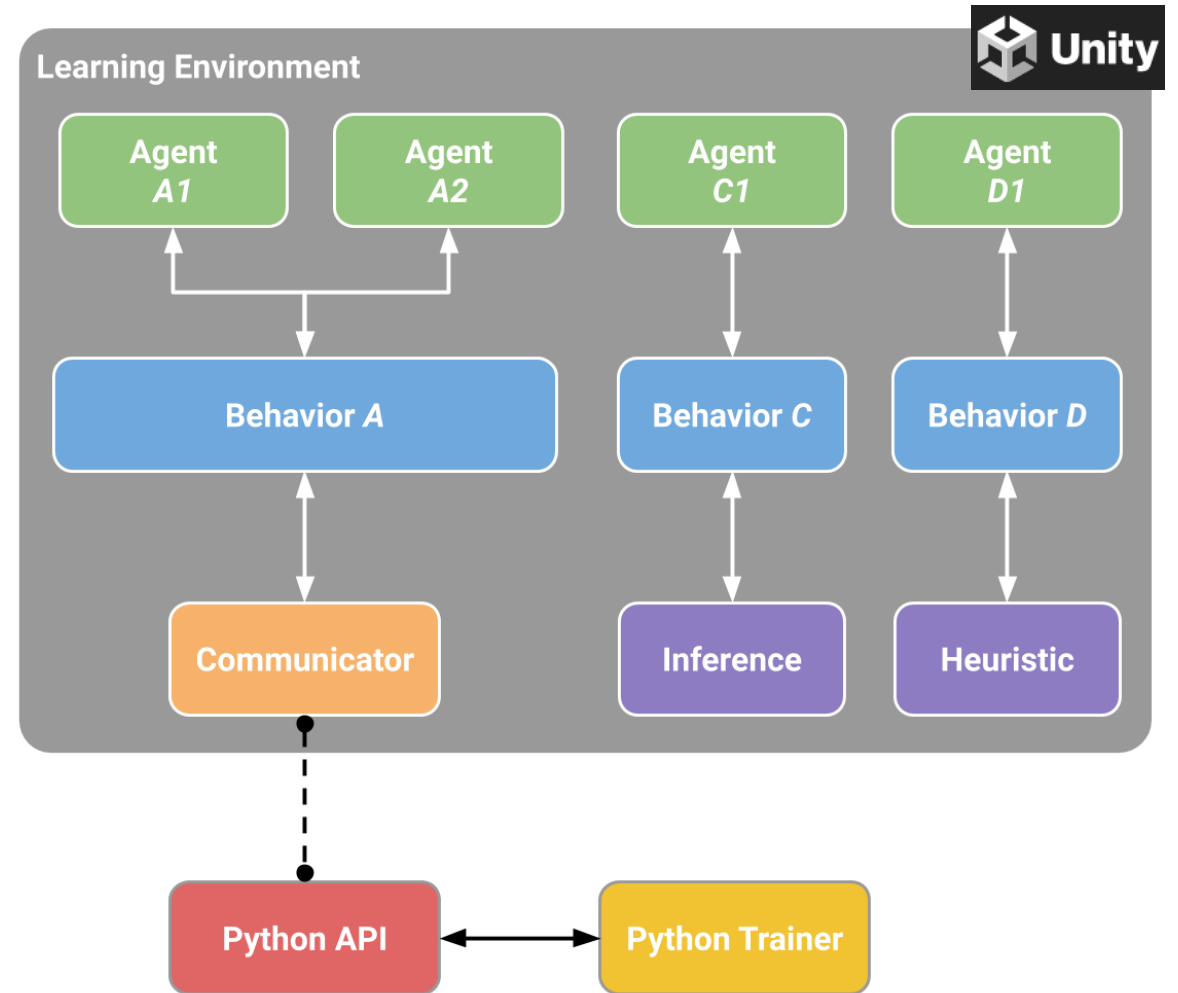
It specifies:

- the shape of the observation space,
- the shape of the action space,
- the name of the behavior — that name is what ties the Unity environment to the Python training run.



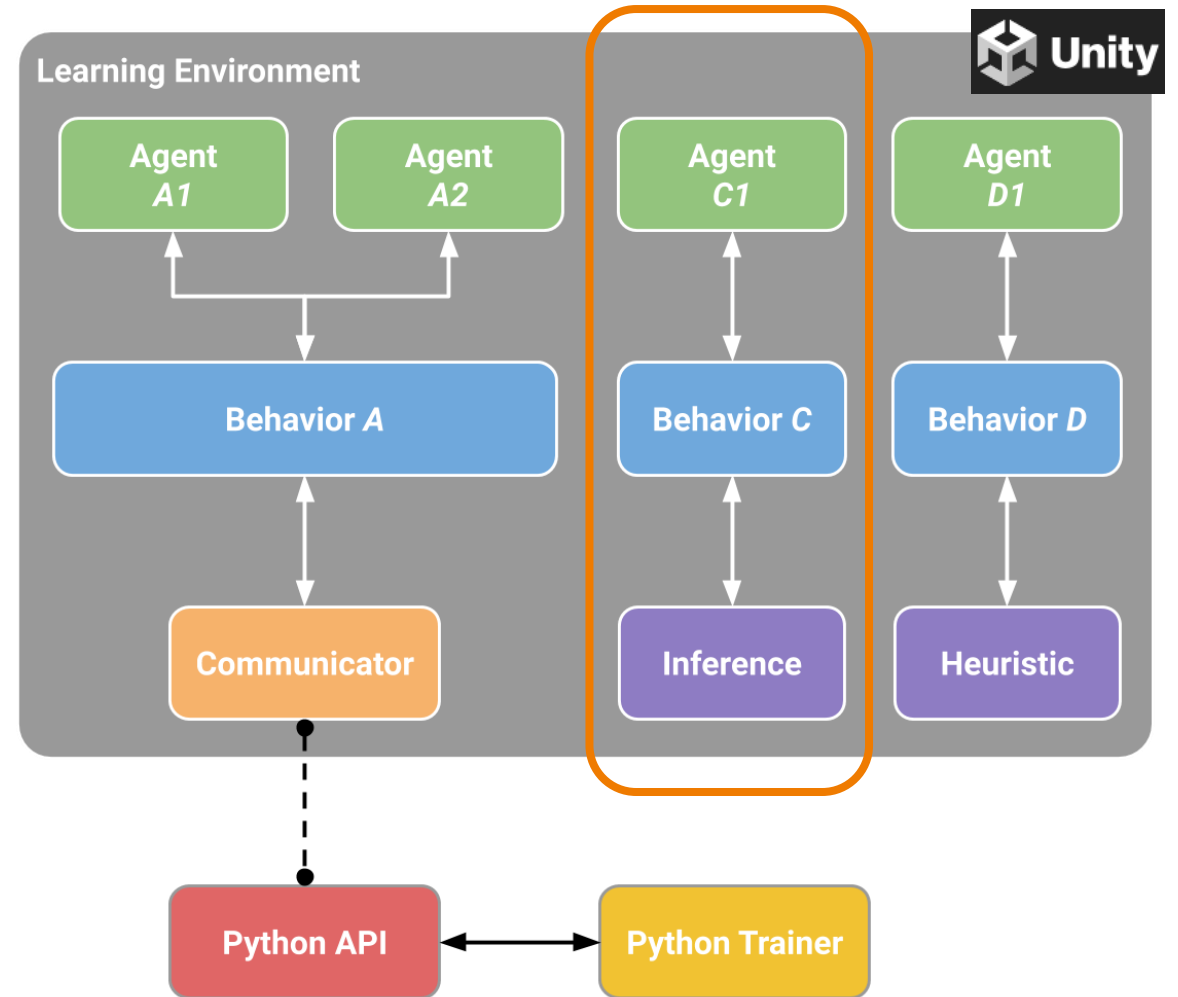
ML-Agents high-level architecture

The **Environment** is just the Unity scene itself. ML-Agents does not require any special scene structure, but the Academy — a singleton that used to be a mandatory component in older versions and is now largely implicit — manages episode resets and environment parameters.



ML-Agents high-level architecture

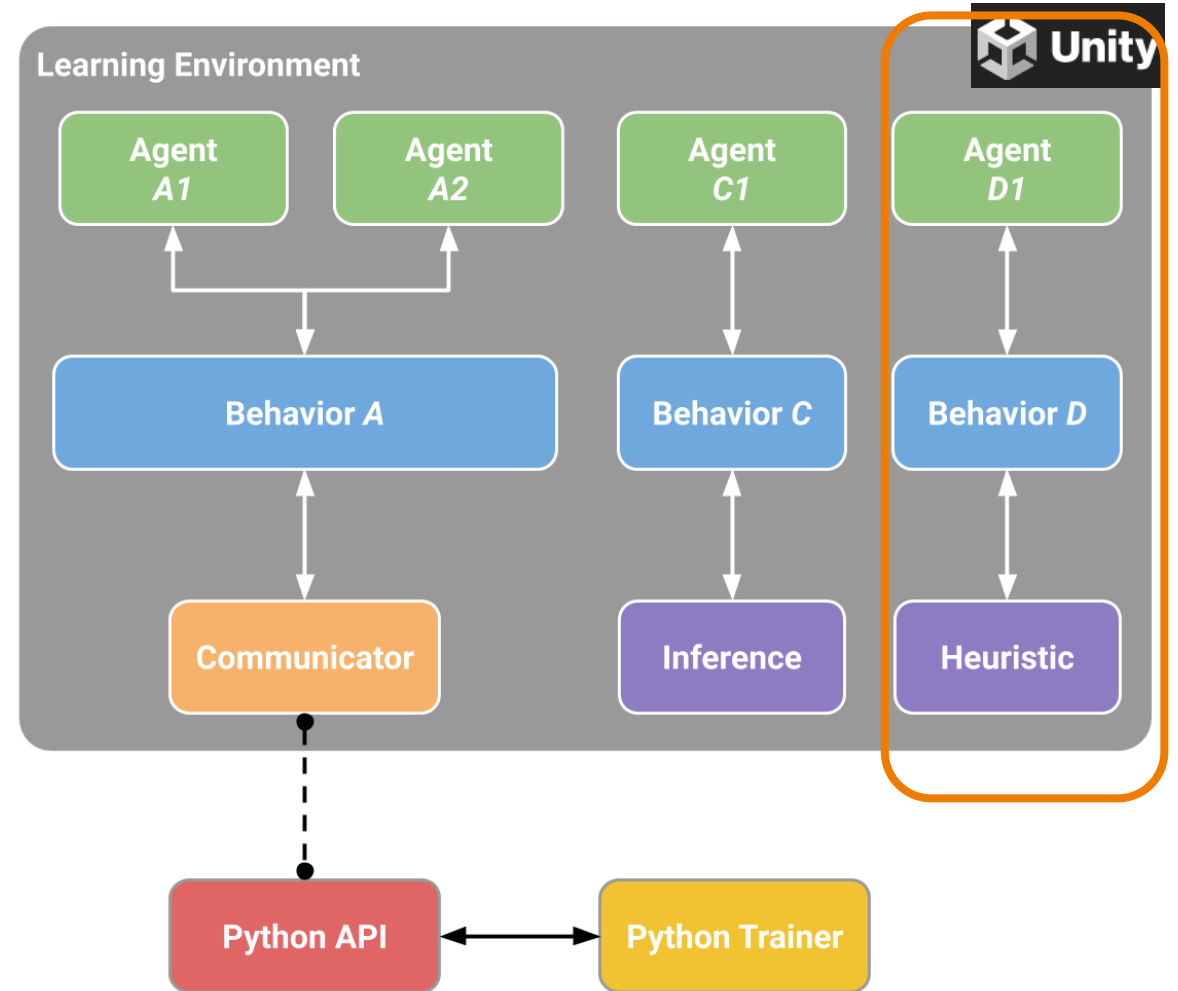
Inference mode: export the trained policy to ONNX format and load it with Unity's Sentis inference engine, so the model runs entirely inside Unity at full game-speed.



ML-Agents high-level architecture

Heuristics: typically used to convert keyword/controller input into actions

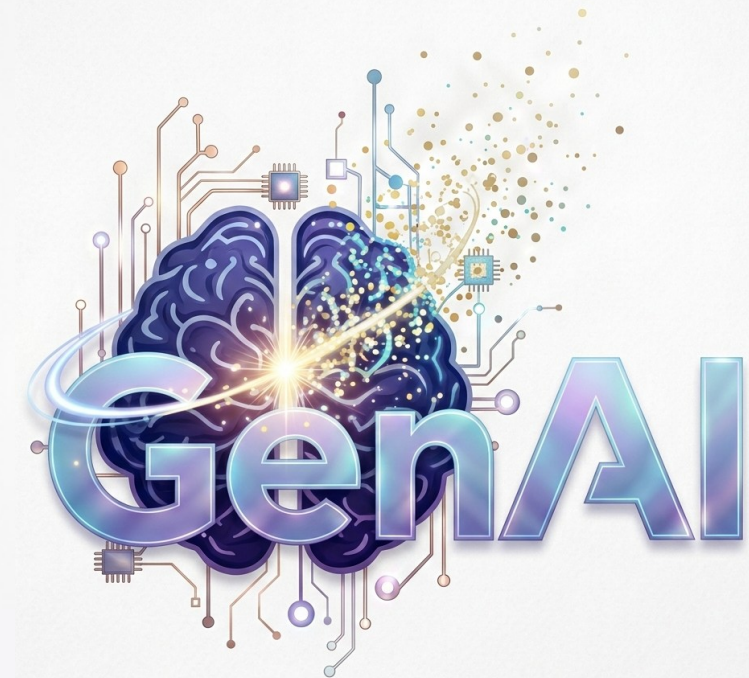
- when the Behavior Type is set to Heuristic Only in the Behavior Parameters of the Agent, the Agent will use the `Heuristic()` method to generate the actions.
- As such, the `Heuristic()` method as argument.
- Used for testing or in multi-player environments





Politecnico
di Torino

Agents, observations and rewards

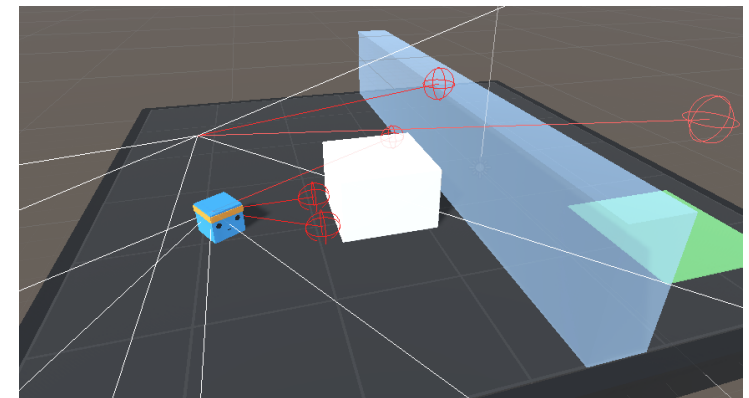
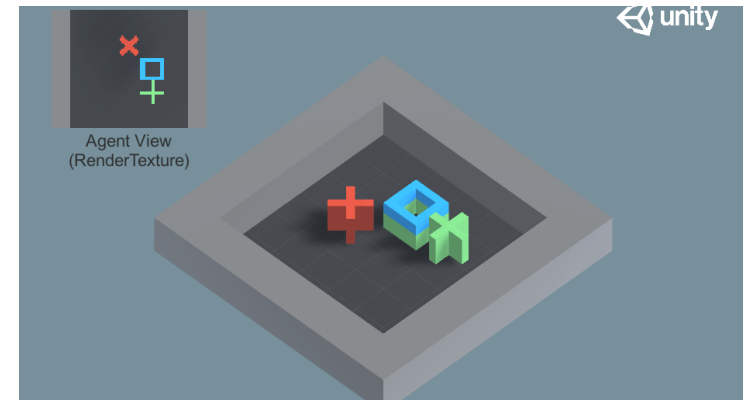


The observation space

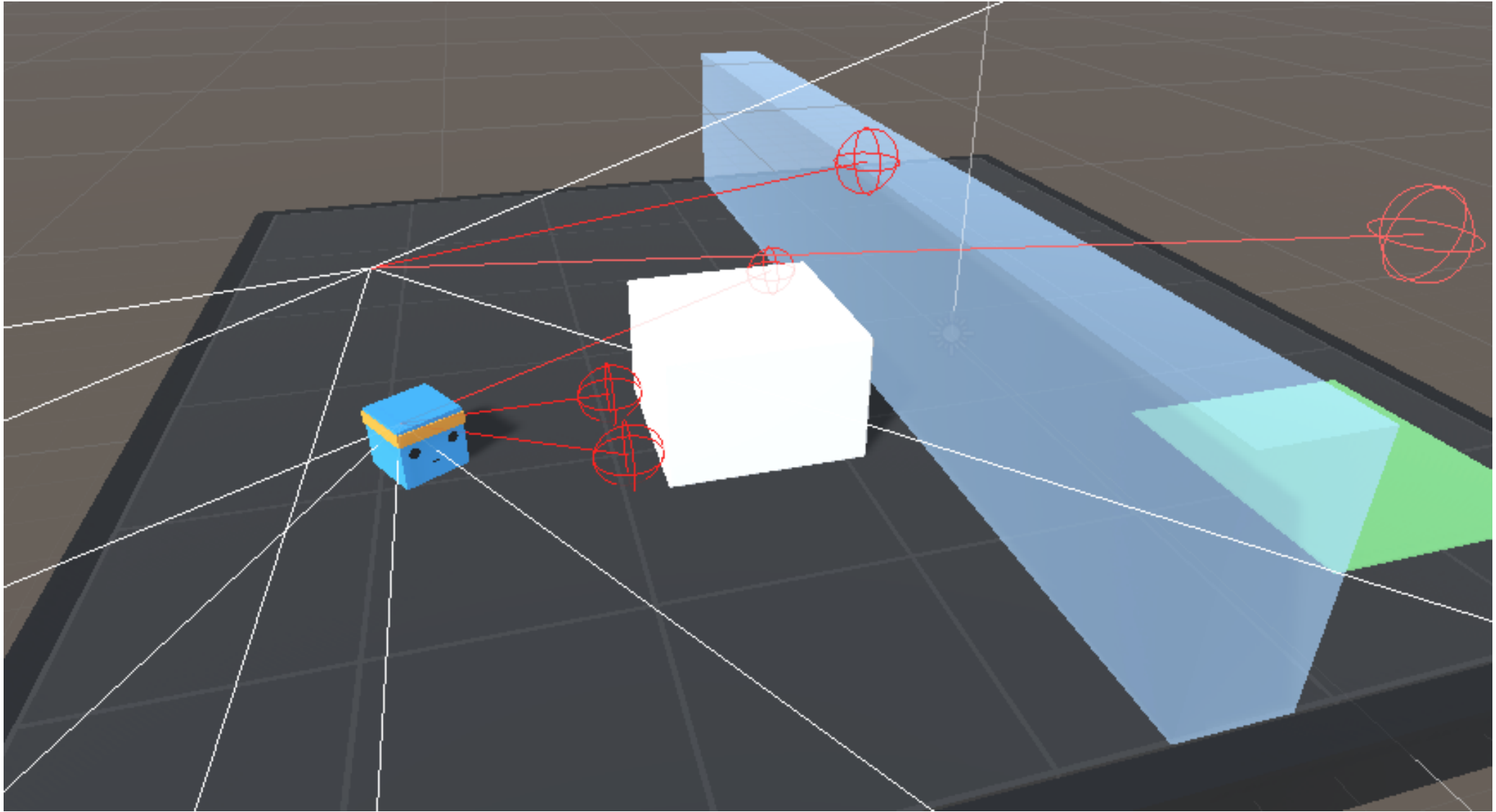
An Agent needs to perceive its world. ML-Agents gives three mechanisms for this:

- **vector observations:** a fixed-size array of floats that you populate manually in the `CollectObservations` method.
- **visual observations:** you attach to the agent and ML-Agents captures a rendered image and feeds it as a pixel tensor to the agent.
- **ray-cast sensors:** a `RayPerceptionSensor` component casts a fan of rays from the agent into the scene and detects what they hit — walls, enemies, goals — along with the distance.

```
step 1: [0.1, 0.0, 0.0]
step 2: [0.2, 0.1, 0.0]
step 3: [0.3, 0.2, 0.1]
step 4: [0.4, 0.3, 0.2]
```

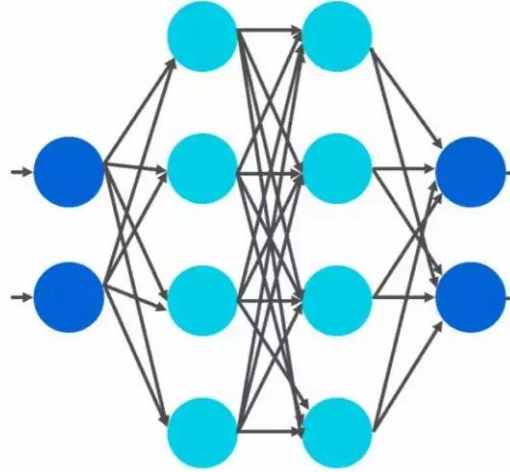


Ray casting

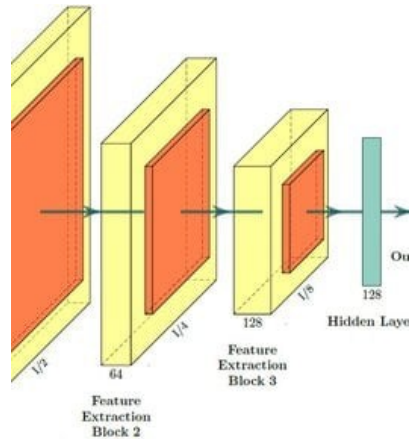
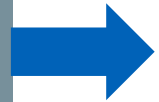
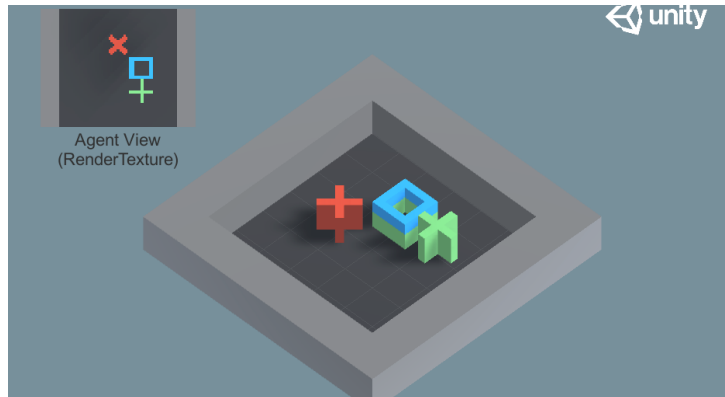


Agent architecture

step 1: [0.1, 0.0, 0.0]
step 2: [0.2, 0.1, 0.0]
step 3: [0.3, 0.2, 0.1]
step 4: [0.4, 0.3, 0.2]



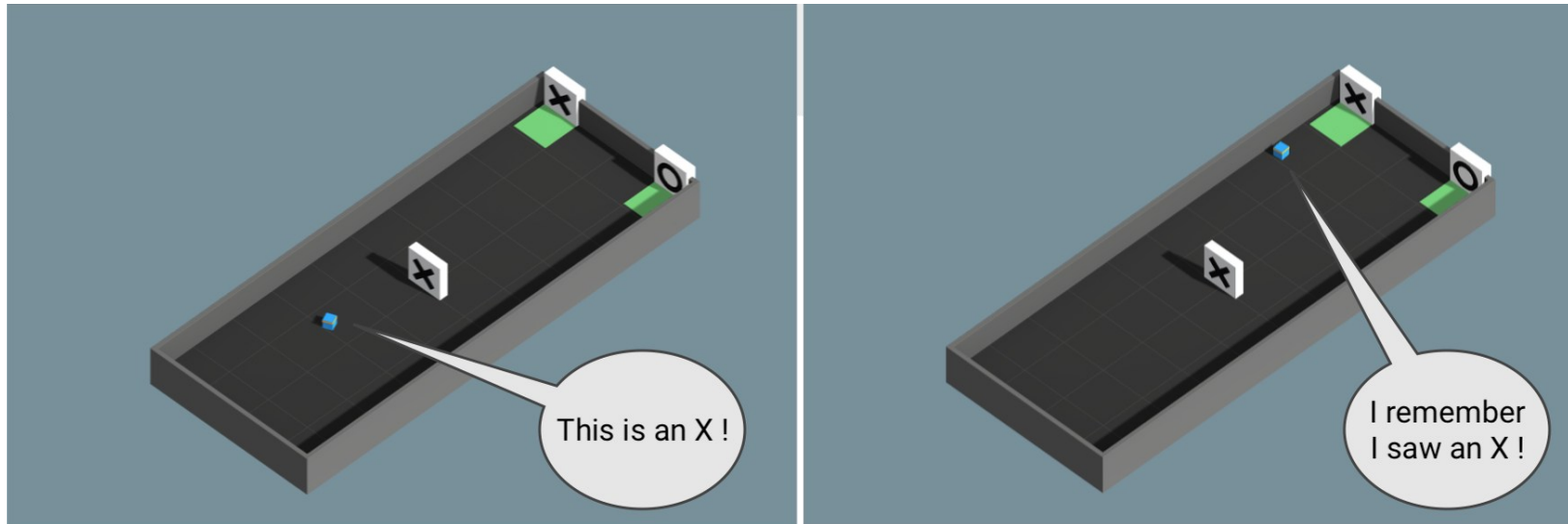
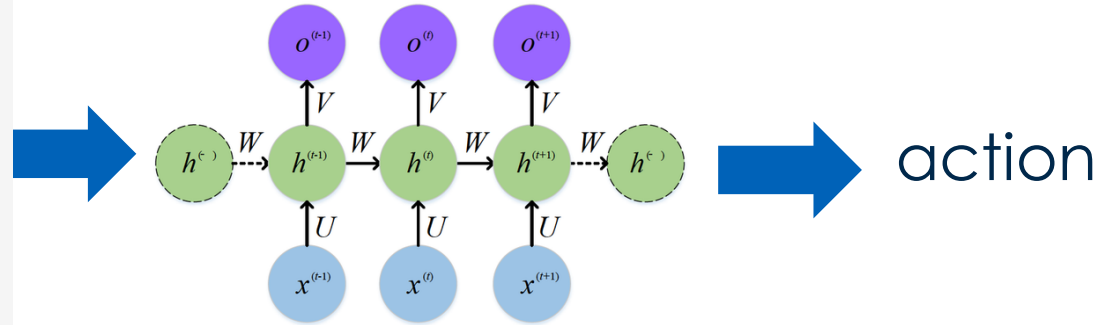
action



action

Memory-augmented agents

step 1: [0.1, 0.0, 0.0]
step 2: [0.2, 0.1, 0.0]
step 3: [0.3, 0.2, 0.1]
step 4: [0.4, 0.3, 0.2]

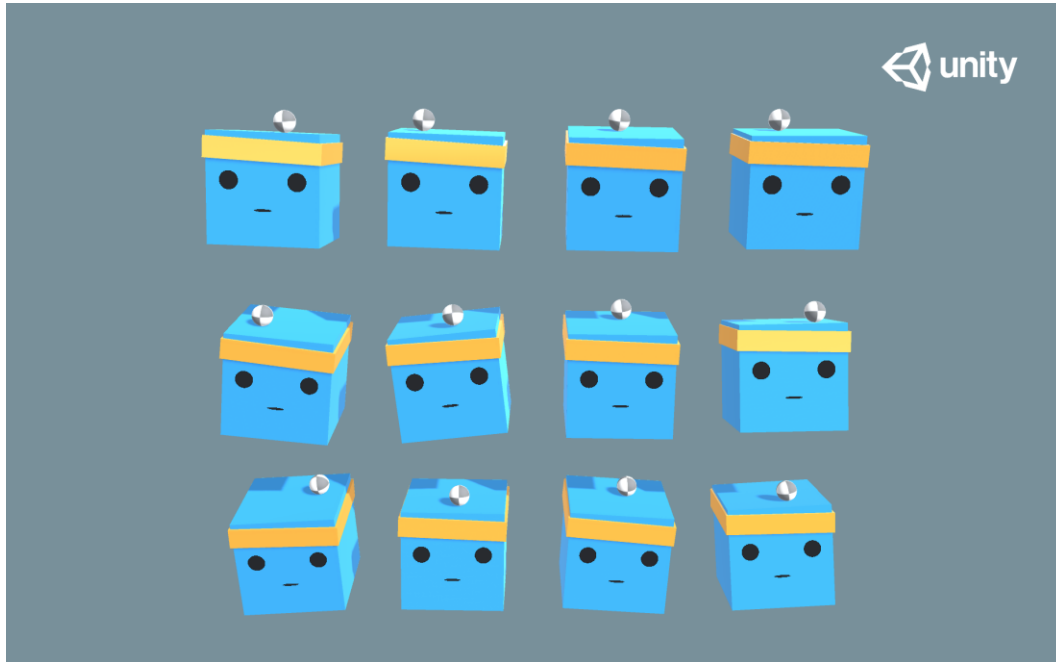


The action space

Actions are what the policy outputs — the decisions the agent makes.

- **Discrete actions** are integer choices. You define **branches**: for example, branch 0 might have three choices (move forward, move backward, stay still) and branch 1 might have three choices (rotate left, rotate right, don't rotate). The agent outputs **one integer per branch per step**. This is appropriate for tasks with a finite set of decisions.
- **Continuous actions** are floating-point values (normally) in the range $[-1, 1]$. You define how many continuous dimensions you need — for example, two values to control forward speed and turn rate simultaneously. The policy outputs all values at once. This is appropriate for, e.g., **smooth, physics-driven** locomotion.

Recap: agent properties



Behavior Parameters

Behavior Name: 3DBall

Vector Observation

Space Size: 8

Stacked Vectors: 1

Actions

Continuous Actions: 2

Discrete Branches: 0

Model: 3DBall (NNModel)

Inference Device: CPU

Deterministic Inference: ☐

Behavior Type: Default

Team Id: 0

Use Child Sensors: ☒

Observable Attribute Handling: Ignore

Ball 3D Agent (Script)

Max Step: 5000

Script: Ball3DAgent

Specific to Ball3D

Ball: Ball

Use Vec Obs: ☒

Example: locomotion in two dimension

Discrete actions:

- one Branch with a size of four (one for each direction)
- two Branches of size two (one for horizontal movement and one for vertical movement)

Continuous actions:

- action size to two (one for each dimension), float values

Mixed actions:

- one continuous action for horizontal movement and a discrete branch of size two for the vertical movement (or viceversa)

You can mix discrete and continuous in the same agent, e.g., navigation tasks often use continuous actions for movement and discrete actions for binary events like jumping or firing.

From action to state

- Neither the Policy nor the training algorithm know anything about what the action values themselves mean.
- The training algorithm simply tries different values for the action list and observes the effect on the accumulated rewards over time and many training episodes.
- Thus, the only place actions are defined for an Agent is in the `OnActionReceived()` function. In this function, actions are applied and rewards assigned.

The Reward Signal

- Rewards accumulate over the course of an episode via the `AddReward(float value)` function.
- `SetReward(float value)` can be used to override rather than accumulate.
- At the end of an episode — whether the agent reached the goal, fell off the platform, or hit the maximum step count — `EndEpisode()` triggers a reset.

The reward signal is the **most important design decision**.

ML-Agents provides no reward by default: it must be implemented in C#.

The Reward signal: example

Common pattern for moving agents is to assign:

- a large positive reward on goal completion (e.g., +1.0),
- a small negative reward every step to incentivize speed (e.g., -0.001)
- and a large negative reward on failure conditions (e.g., -1.0 on falling).

The exact values matter less than their **ratios** — what you're shaping is **relative incentive**.

Type of rewards

Generated by the environment

- Extrinsic

Generated by the agent (controlled via configuration file – see training later)

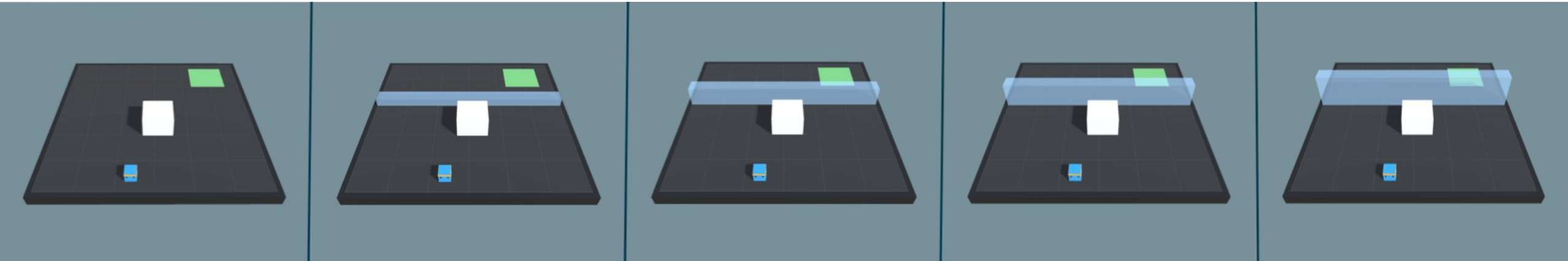
- GAIL intrinsic
- Curiosity
- Random Network Distillation (RND)

Academy

- The Academy is a global singleton in ML-Agents.
- Its most useful feature for teaching purposes is **EnvironmentParameters**: named float values you can set from the Python trainer's YAML configuration and read from C# at runtime.
- Environment parameters enable advanced training strategies like curriculum learning:
 - you define a difficulty parameter like `obstacle_density` and your C# code reads it to configure the scene.
 - the trainer then increases it automatically as the agent improves.

Curriculum learning

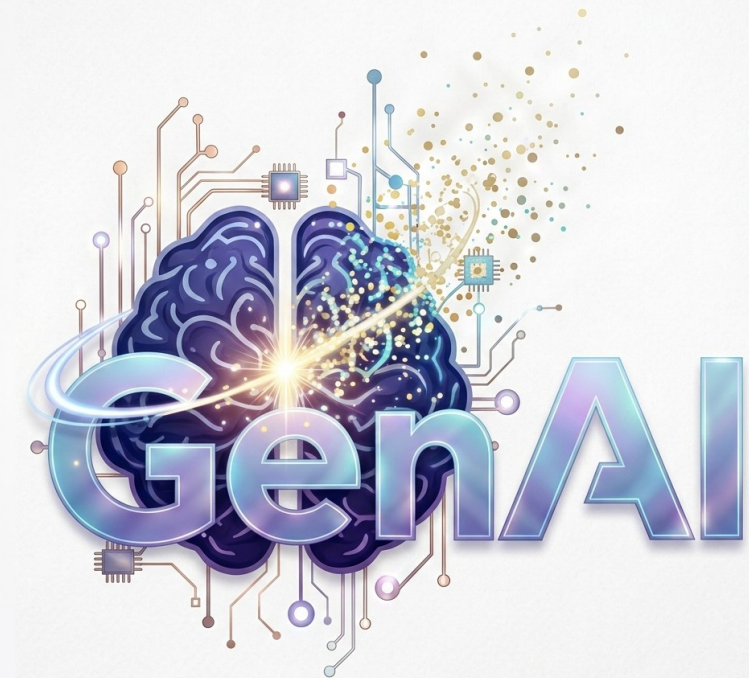
Demonstration of a hypothetical curriculum training scenario in which a progressively taller wall obstructs the path to the goal.





Politecnico
di Torino

Customizing Agents: an example

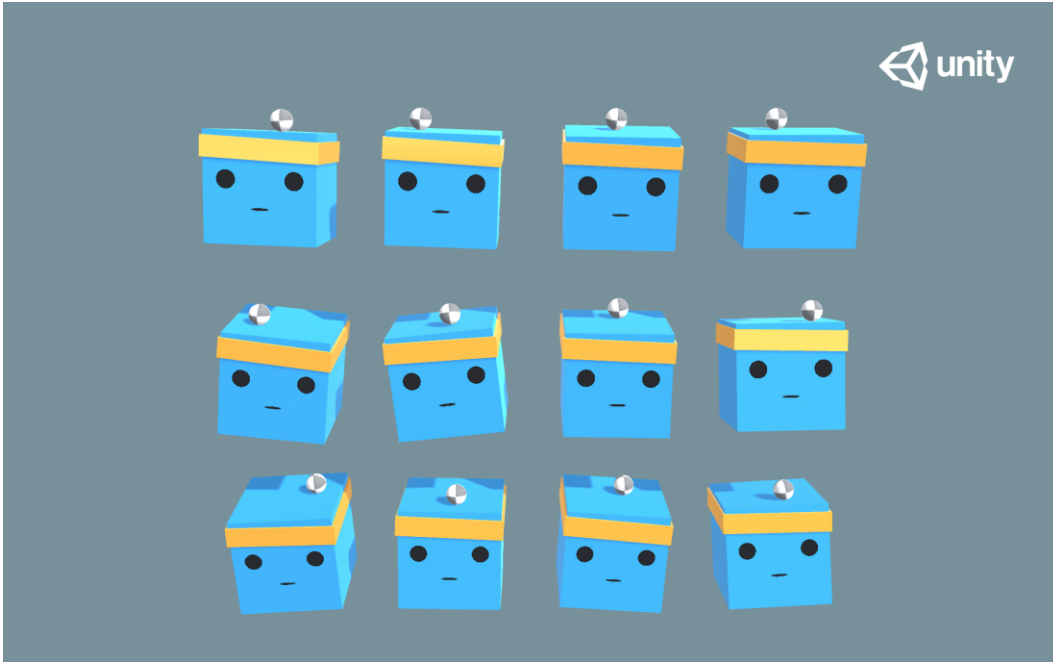


Creating a trainable agent

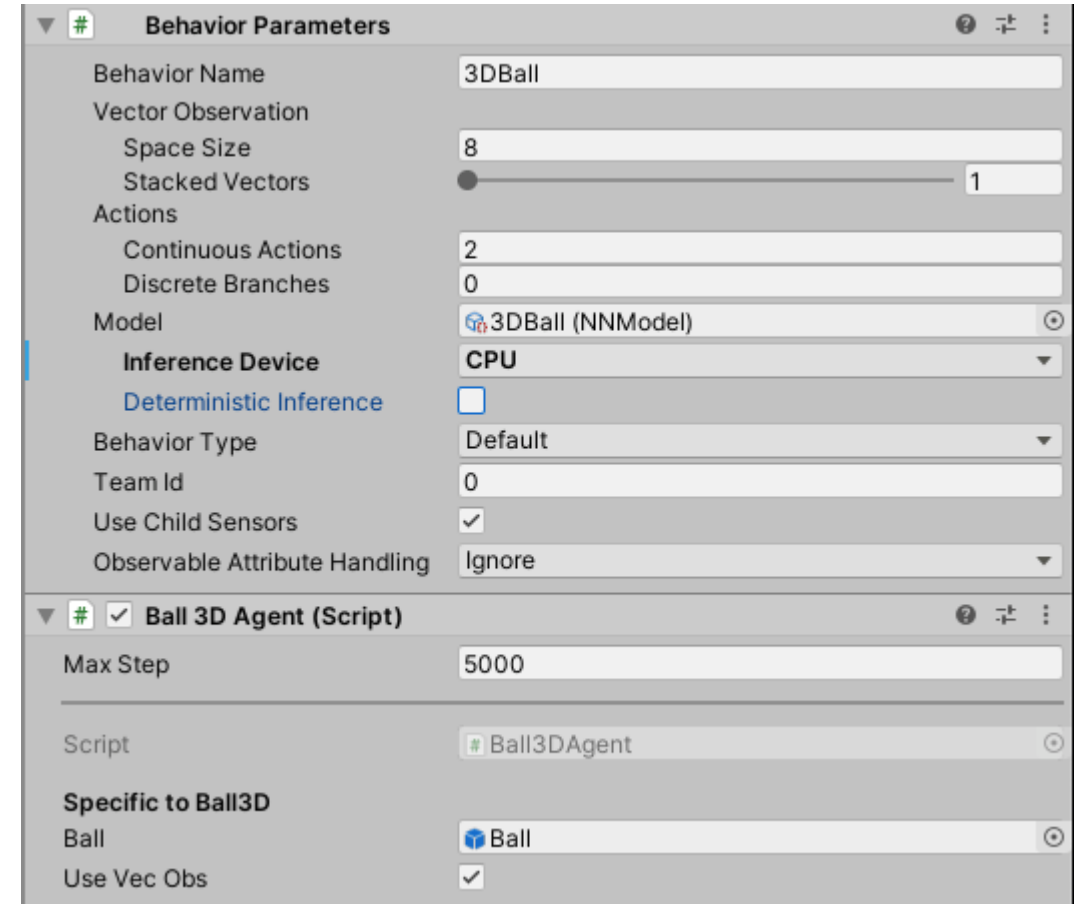
When you create an Agent, you should usually extend the base `Agent` class. This includes implementing the following methods:

- `Agent.OnEpisodeBegin()` — Called at the beginning of an Agent's episode, including at the beginning of the simulation.
- `Agent.CollectObservations(VectorSensor sensor)` — Called every step that the Agent requests a decision. This is one possible way for collecting the Agent's observations of the environment. There are more sophisticated alternatives (e.g. using `ISensor` interface)
- `Agent.OnActionReceived(ActionBuffers actions)` — Called every time the Agent receives an action to take. Receives the action chosen by the Agent. It is also common to assign a reward in this method.
- Reward could also be assigned in other methods, e.g., `OnCollisionEnter(Collision collision)`, associated with specific events in Unity
- `Agent.Heuristic()` — the Agent will use the `Heuristic()` method to generate the actions of the Agent. As such, it writes to the array of floats provided to the method as argument.

Sample agent



The C# agent script is found under `Assets/ML-Agents/Examples/3DBall/Scripts/Ball3DAgent.cs` — this is where all reward and observation modifications go.



Reset the episode

```
public override void OnEpisodeBegin()
{
    gameObject.transform.rotation = new Quaternion(0f, 0f, 0f, 0f);
    gameObject.transform.Rotate(new Vector3(1, 0, 0), Random.Range(-10f, 10f));
    gameObject.transform.Rotate(new Vector3(0, 0, 1), Random.Range(-10f, 10f));
    m_BallRb.linearVelocity = new Vector3(0f, 0f, 0f);
    //Position the ball in a random position
    ball.transform.position = new Vector3(Random.Range(-1.5f, 1.5f), 4f,
                                           Random.Range(-1.5f, 1.5f))
                                   + gameObject.transform.position;
    //Reset the parameters when the Agent is reset.
    SetResetParameters();
}
```

Collect observation

```
public GameObject ball;

public override void CollectObservations(VectorSensor sensor)
{
    // Orientation of the cube (2 floats)
    sensor.AddObservation(gameObject.transform.rotation.z);
    sensor.AddObservation(gameObject.transform.rotation.x);
    // Relative position of the ball to the cube (3 floats)
    sensor.AddObservation(ball.transform.position - gameObject.transform.position);
    // Velocity of the ball (3 floats)
    sensor.AddObservation(m_BallRb.velocity);
    // 8 floats total
}
```

Implement action – continuous action

```
public override void OnActionReceived(ActionBuffers actionBuffers)
{
    //Clip the action between -1 and 1
    var actionZ = 2f * Mathf.Clamp(actionBuffers.ContinuousActions[0], -1f, 1f);
    var actionX = 2f * Mathf.Clamp(actionBuffers.ContinuousActions[1], -1f, 1f);

    //Rotate the cube
    gameObject.transform.Rotate(new Vector3(0, 0, 1), actionZ);
    gameObject.transform.Rotate(new Vector3(1, 0, 0), actionX);
}
```

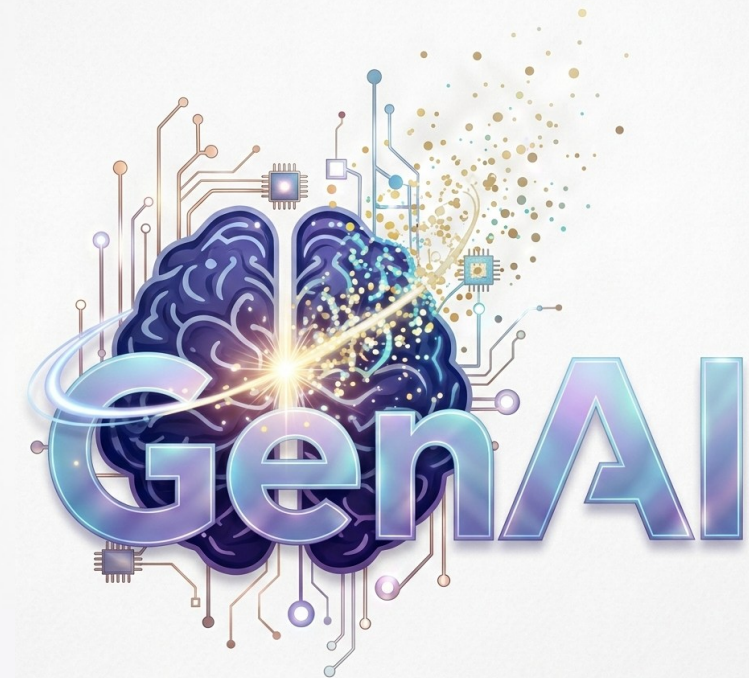
Implement action – set reward

```
if ((ball.transform.position.y - gameObject.transform.position.y) < -2f ||  
    Mathf.Abs(ball.transform.position.x - gameObject.transform.position.x) > 3f ||  
    Mathf.Abs(ball.transform.position.z - gameObject.transform.position.z) > 3f)  
{  
    // if the ball falls out of the cube, assign negative reward  
    SetReward(-1f);  
    EndEpisode();  
}  
else  
{    // positive reward for keeping the game alive  
    SetReward(0.1f);  
}  
}
```



Politecnico
di Torino

The training side



Training

- `mlagents-learn` is the **main training utility** provided by the ML-Agents Toolkit. It accepts a number of CLI options in addition to a YAML configuration file that contains all the configurations and hyperparameters to be used during training.

```
mlagents-learn <trainer-config-file> --env=<env_name> --run-id=<run-identifier>
```

Training config file options: <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/Training-Configuration-File.html>

- ML-Agents provides support for running custom python implementations of specific interfaces during the training process via Plugin. **Note:** Plugin interfaces should currently be considered "in beta", and they may change in future releases.

Available algorithms (1/2)

- **PPO**: default algorithm in ML-Agents and the right starting point for almost every new task. It is an on-policy algorithm, which means it collects experience with the current policy and then discards it after a few gradient updates. This makes it sample-inefficient but very stable.
- **SAC**: an off-policy algorithm, meaning it can reuse experience from a replay buffer rather than discarding it. This makes it significantly more sample-efficient than PPO for continuous action tasks but harder to stabilize. It also has a built-in entropy maximization objective, which encourages exploration without a separate entropy coefficient to tune.

Prefer SAC when your action space is continuous, when you have limited simulation budget (fewer steps available), or when PPO plateaus.

Available algorithms (2/2)

- **GAIL** allows you to **train an agent from demonstrations** — recordings of a human or scripted agent completing the task — without writing an explicit reward function. A discriminator neural network learns to distinguish the agent's behavior from the expert's behavior. The agent's reward is derived from fooling this discriminator.
- This is powerful in tasks where reward engineering is difficult. Record demonstrations using **Unity's built-in demonstration recorder** component, save them as .demo files, and reference them in the YAML configuration under the `behavioral_cloning` and `gail` reward signal sections.
- **BC** is simpler than GAIL: it is supervised learning on demonstrations. The policy is initialized by minimizing the difference between its outputs and the expert's actions in the recorded demonstrations. It cannot go beyond the expert's performance, and it **fails in states not covered by demonstrations**,

Example YAML file

PPO hyperparameters

Neural network size

Reward

```
behaviors:
  NavigationBehavior:
    trainer_type: ppo
    hyperparameters:
      batch_size: 128
      buffer_size: 2048
      learning_rate: 3.0e-4
      beta: 5.0e-3
      epsilon: 0.2
      lambda: 0.95
      num_epoch: 3
      learning_rate_schedule: linear
    network_settings:
      normalize: true
      hidden_units: 128
      num_layers: 2
    reward_signals:
      extrinsic:
        gamma: 0.99
        strength: 1.0
      max_steps: 500000
      time_horizon: 64
      summary_freq: 10000
```

config/ppo/3DBall.yaml

YAML configuration file (PPO)

`batch_size` is the number of experiences used in each gradient update. Larger batches produce more stable gradient estimates but require more memory. For continuous action spaces, 512–2048 is typical; for discrete, 32–512.

`buffer_size` is how much experience you collect before you start updating at all. This should always be a multiple of `batch_size`. A larger buffer means more diverse experience per update cycle.

`learning_rate` controls the step size of gradient updates. Default is $3e-4$. Too high and training diverges; too low and it crawls. A linear or exponential decay schedule is often useful.

`entropy_coeff` encourages exploration by penalizing overly confident action distributions. If the agent converges too early to a suboptimal behavior, increasing this helps it keep exploring.

`clip_param` is the PPO clipping ratio, the core of the algorithm. It constrains how much the policy can change in a single update, preventing destructive large steps. Default is 0.2.

`lambda` is the GAE (Generalized Advantage Estimation) parameter. It trades off between high-variance Monte Carlo returns ($\lambda=1$) and high-bias bootstrapped TD estimates ($\lambda=0$). 0.95 is a good default.

Monitoring training

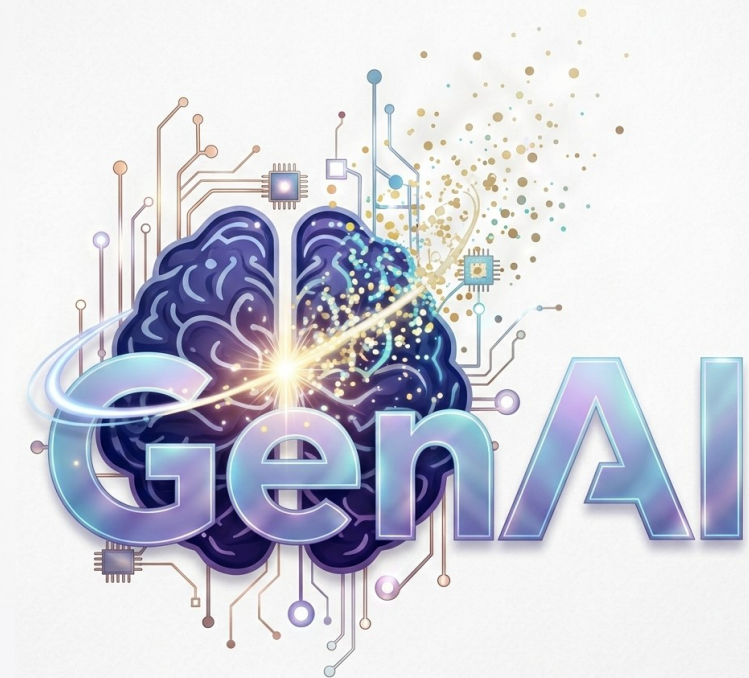
Regardless of training method and configurations, the training process will always generate three artifacts, all found in the results/<run-identifier> folder:

- **Summaries:** these are training metrics that are updated throughout the training process and can be visualized using Tensorboard.
- **Models:** these contain the model checkpoints that are updated throughout training and the final model file (.onnx). This final model file is generated once either when training completes or is interrupted.
- **Timers** file (under results/<run-identifier>/run_logs): aggregated metrics on your training process, including time spent on specific code blocks. See for more information on the timers generated.
- These artifacts are updated throughout the training process and finalized when training is completed or is interrupted.

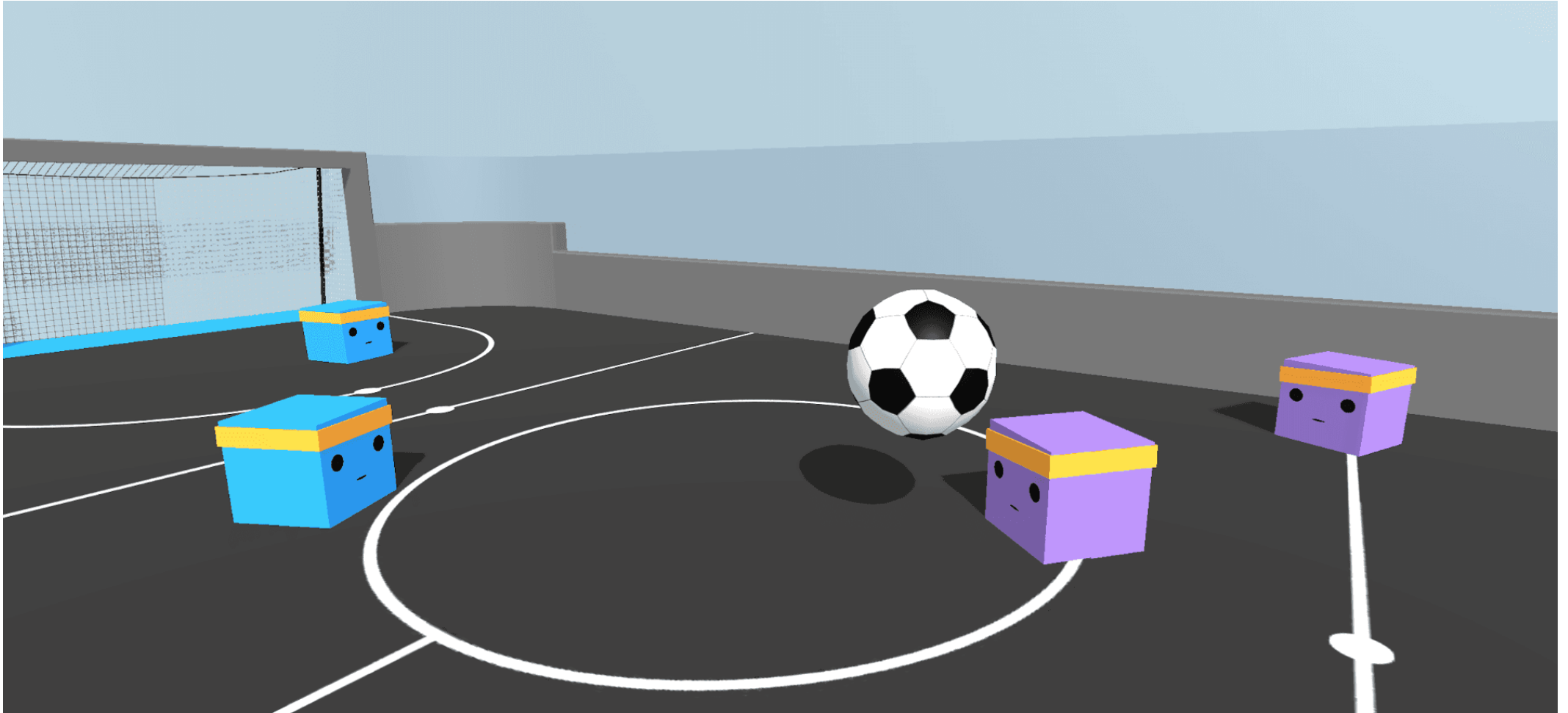


Politecnico
di Torino

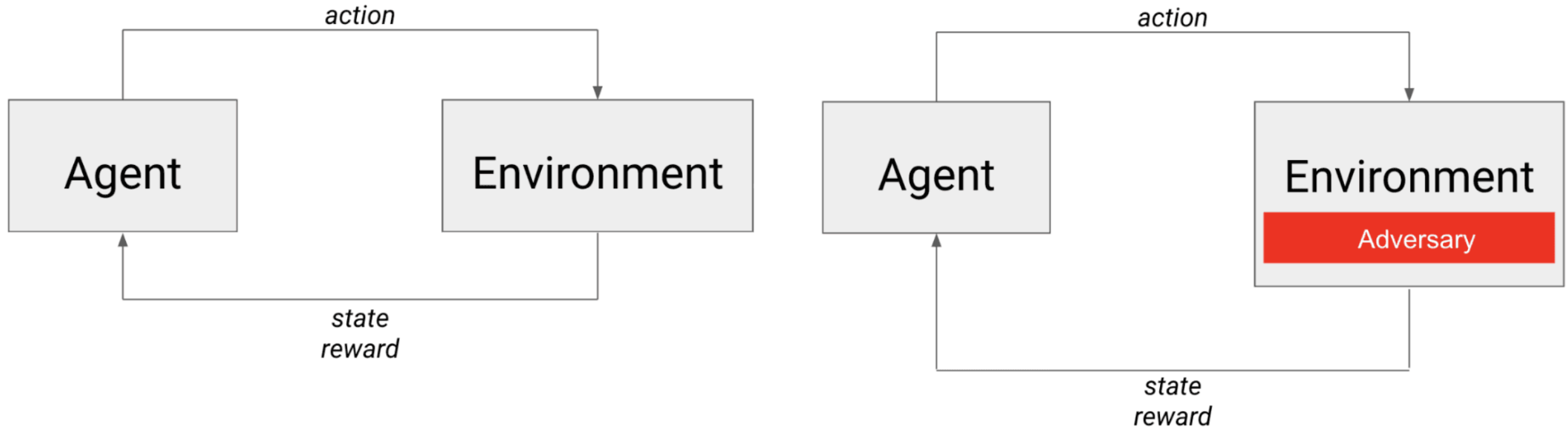
Multi-agents scenarios



Multi-agents scenarios

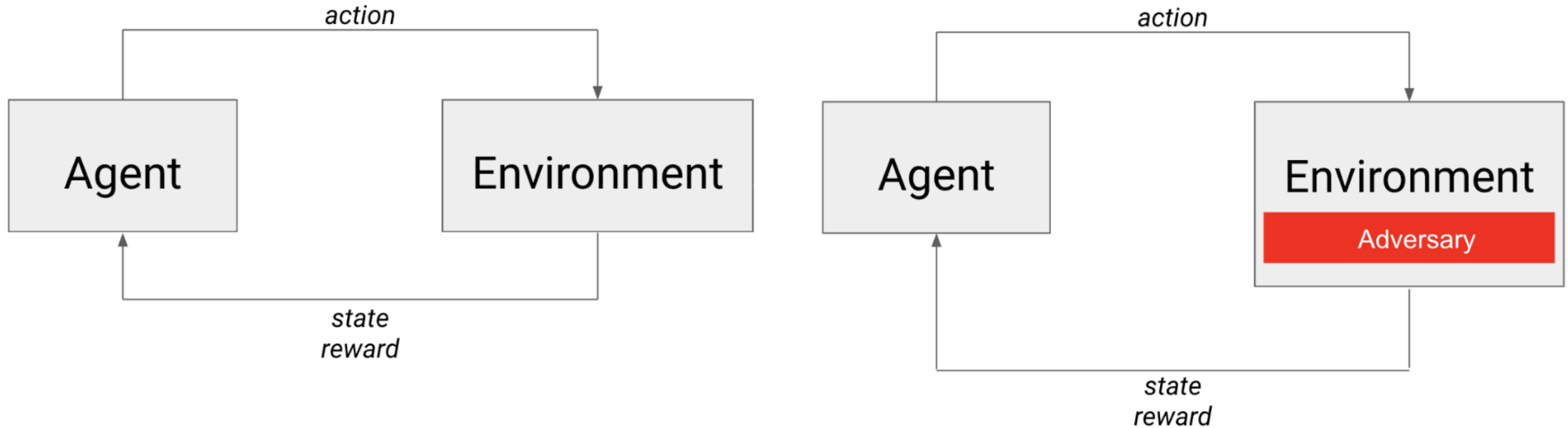


Adversarial games



When the agent competes with an adversary: from the agent's perspective, the agent is effectively part of the environment.

Self play to the rescue



The *agent itself* satisfies both requirements for a fitting opponent. It is certainly roughly equal in skill (to itself) and also improves over time.

Multi agents: training vs. playing phase

